Faculty of Engineering & Technology

Electrical & Computer Engineering Department

Computer Architecture

ENCS4370

Project #2 report

Design and Verification of a Simplified Predicated RISC Processor using Verilog

---

Prepared by:

Batol Abu Samhadana	1230738	Section 3
Hala Sarsour	1230058	Section 1

Instructors:

Dr. Ayman Hroub

Dr. Ahmad Afana

Date: January 9th, 2025

Abstract

Thin project focuses on designing and building a simplified predicated RISC processor that executes 32-bit instructions. The processor includes 32 general-purpose registers, including registers dedicated for the program counter and return address. It also uses separate instruction and data memories with word addressable access. The program supports predicated execution, allowing conditional instruction execution based on the value of a predicate register. It handles R-Type, I-Type and J-Type instructions, covering arithmetic, logic, memory and jump operations.

The datapath includes multiple stages (fetch, decode,execute, memory write and read, and write-back). And the controller generates signals based on truth tables, Boolean equations and state diagrams. The simulation we did using testbench verifies the processor with multiple instructions. In the rest of this report we will cover the details of the design, implementation choices, test cases and simulation.

Table of Contents

Table of Figures

List of tables

1. Design Specifications

1.1. Instruction formats

1.1.1. R-Type

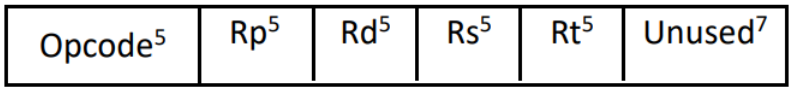


Figure 1 R-Type instruction formulation

Rp: Predicate register

Rd: Destination register

Rs: First operand register

Rt: Second operand register

1.1.2. I-Type

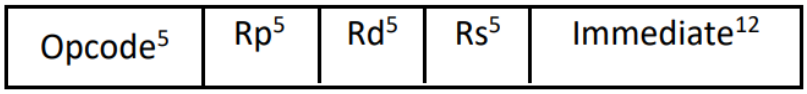


Figure 2 I-Type instruction formulation

Rp: Predicate register

Rd: Destination register

Rs: First operand register

Imm: The numerical value, unsigned for logical instructions and signed arithmetic instructions.

1.1.3. J-type

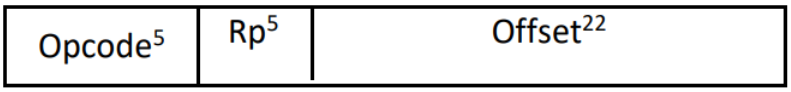


Figure 3 J-Type instruction formulation

Rp: Predicate register

Offset: The amount added to the current address to jump to the target address.

1.2. Instruction Set

Table 1 instruction set encoding

Instruction	Type	Meaning (RTN)	Opcode
ADD Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] ≠ 0) → Reg[Rd] = Reg[Rs] + Reg[Rt]	0
SUB Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] ≠ 0) → Reg[Rd] = Reg[Rs] − Reg[Rt]	1
OR Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] ≠ 0) → Reg[Rd] = Reg[Rs] | Reg[Rt]	2
NOR Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] ≠ 0) → Reg[Rd] = ~(Reg[Rs] | Reg[Rt])	3
AND Rd, Rs, Rt, Rp	R-Type	If (Reg[Rp] ≠ 0) → Reg[Rd] = Reg[Rs] & Reg[Rt]	4
ADDI Rd, Rs, Imm, Rp	I-Type	If (Reg[Rp] ≠ 0) → Reg[Rd] = Reg[Rs] + Imm	5
ORI Rd, Rs, Imm, Rp	I-Type	If (Reg[Rp] ≠ 0) → Reg[Rd] = Reg[Rs] | Imm	6
NORI Rd, Rs, Imm, Rp	I-Type	If (Reg[Rp] ≠ 0) → Reg[Rd] = ~(Reg[Rs] | Imm)	7
ANDI Rd, Rs, Imm, Rp	I-Type	If (Reg[Rp] ≠ 0) → Reg[Rd] = Reg[Rs] & Imm	8
LW Rd, Imm(Rs), Rp	I-Type	If (Reg[Rp] ≠ 0) → Reg[Rd] = Mem(Reg[Rs] + Imm)	9
SW Rd, Imm(Rs), Rp	I-Type	If (Reg[Rp] ≠ 0) → Mem(Reg[Rs] + Imm) = Reg[Rd]	10

J Label, Rp	J-Type	If (Reg[Rp] ≠ 0) → Jump to target address	11
CALL Label, Rp	J-Type	If (Reg[Rp] ≠ 0) → Jump to function, store return address in R31	12
JR Rs, Rp	R-Type	If (Reg[Rp] ≠ 0) → Jump to address in Reg[Rs]	13

2. Datapath Components

2.1. Multiplexer

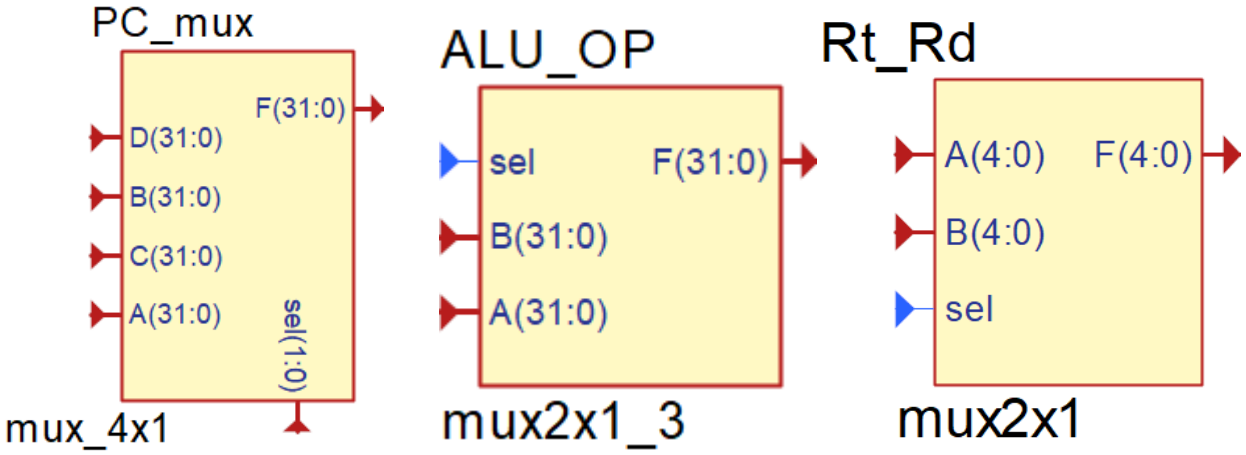


Figure 4 Different Multiplexers

The multiplexer is very crucial to our design. It selects between several inputs and forwards the selected input to a single output line. Two types of multiplexers were utilized in our design: the 4x1 mux and 2x1 mux. The size of the mux was determined based on the number of bits of the select inputs.

For example:

When choosing the next PC value to be written on the PC register based on the PC_src control signal a 4x1 mux was used since the PC_src is a 2 bit control signal.

When PC_src is set to 00 by the controller, the next PC value will be the current PC value incremented by 1.

When PC_src is set to 01 by the controller, the next PC value will be the jump address calculated by the jump_calc module which essentially adds the current PC value to the sign extended offset value.

When PC_src is set to 10 by the controller, the next PC value will be the value of Reg[Rs].

2.2. Instruction Memory

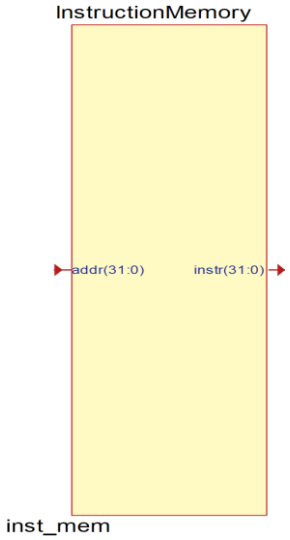


Figure 5: Instruction memory module

The design implements what is known as the Harvard architecture where the instructions and data are stored in separate memories. In the instruction memory, there is only a singular read port because the datapath does not write instructions. The 32-bit address of the instruction arrives from the PC located in the register and the memory outputs a 32-bit.

2.3. Data Memory

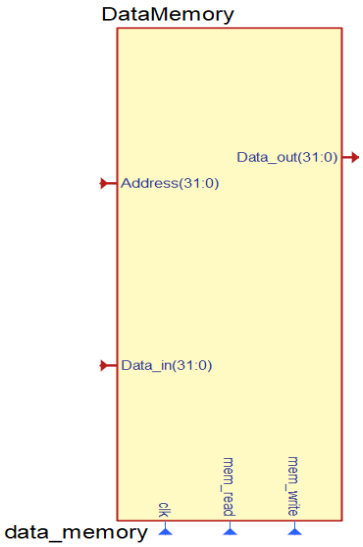


Figure 6 Data memory module

As aforementioned, the Harvard architecture utilizes separate memories for data and instructions.

The data stored in the data memory can only be accessed in load and store instruction as is custom for RISC architecture. It is controlled by two control signals: mem_read, and mem_write. The mem_read input enables and disables the output; meaning the data can only be read when mem_read is activated. The mem_Write enables the writing of Data_in; meaning that data can only be stored when mem_write is active. Additionally, the reading data from the memory was designed to be asynchronous which entails that data can be read regardless of the clock cycle. On the other hand, the writing of data is synchronous which means that it can be performed on the positive edge level of the clock. Moreover, the 32 bit input address selects the data written or read.

2.4. Register File

The register file consists of 32 32-bit registers. R0 is hardwired to zero, meaning it reads zero and cannot be written on. Moreover, R30 is hardwired to the PC and R31 is hardwired to the return address register. The register has three general 32-bit read ports: BusA, BusB, and BusP.

BusA outputs the contents of RA.

BusB outputs the contents of RB.

BusP outputs the contents of Rp.

Where RA, RB, and Rp are 5 bit inputs.

The extra read port was added to access the contents of Rp and perform the predicated execution. Moreover, the register file has a read port that is hardwired to the PC; meaning it only outputs the contents of register 30.

There is a general write port and a write port hardwired to the PC. The general write port is controlled by the reg_write control signal that enables and disables the writing of BusW on the register selected by RW. While PC_write enables and disables the writing of inPC on register 30. Both writing operations are synchronous, meaning writing on the register file can only be done at the positive edge of the clock.

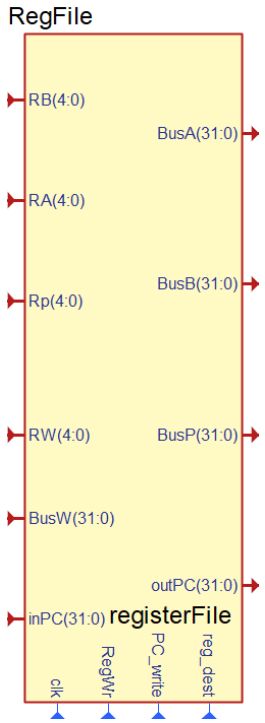


Figure 7 register file module

2.5. Arithmetic Logic Unit (ALU)

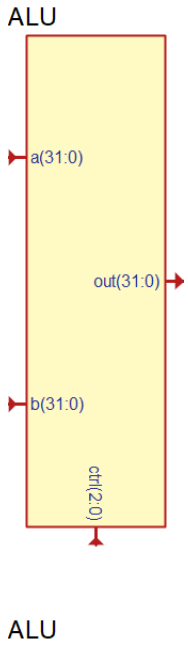


Figure 8 ALU module

The ALU is an important component of the CPU that is responsible for all the arithmetic and logical operations like addition, subtraction and logical comparisons.

In our implementation the ALU takes two 32-bit inputs and selects which operation to apply on them based on the ALU_ctrl signal coming from the controller. We implemented the applications of these operations using simple verilog instructions as such:

```
module ALU (
    input wire [31:0] a, b,
    input wire [2:0] ctrl,
    output reg [31:0] out
    );

    always @(*) begin
        case (ctrl)
            3'b000: out = a + b;  // add, addi
            3'b001: out = a - b;  // sub
            3'b010: out = a | b;  // or, ori
            3'b011: out = ~(a | b);   // nor, nori
            3'b100: out = a & b;    // and, andi
            default: out = 32'b0;
        endcase
    end
endmodule
```

After that it outputs a result that is then entered into the ALU register to store it.

2.6. Jump_Calc

Figure 9 jump_calc module

This module is a vital one in the datapath, specifically for the J-Type instructions. It takes the PC value, and the offset value. It then sign-extends the offset value from 22-bits to 32-bits. And lastly it adds that extended number to the PC value and decrements by one to undo the PC+1 that occurs in the prior stage, jumping to the exact right address.

2.7. Comparator

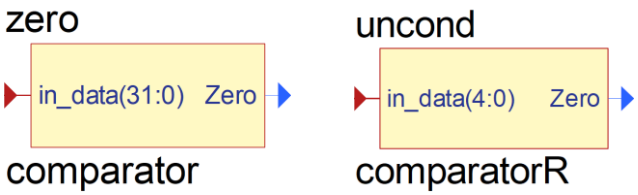


Figure 10 Comparator modules

We used the comparator 2 times in our datapath to implement the predicate specifications. The first time to compare the number of the register Rp with R0 to apply the unconditional predicate part. The second time was to compare the BUS value of the Rp register with 0 and only allow the instruction execution if they're not equal.

2.8. PC increment

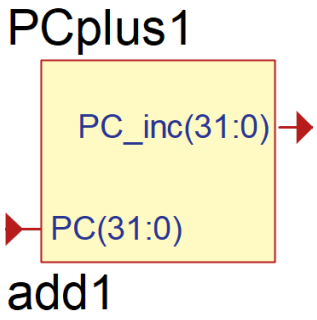


Figure 11 PC adder module

This simple module only takes the current PC and adds 1 to it, we did this in a separate module to make it simpler and to not overcomplicate the ALU logic.

2.9. Registers

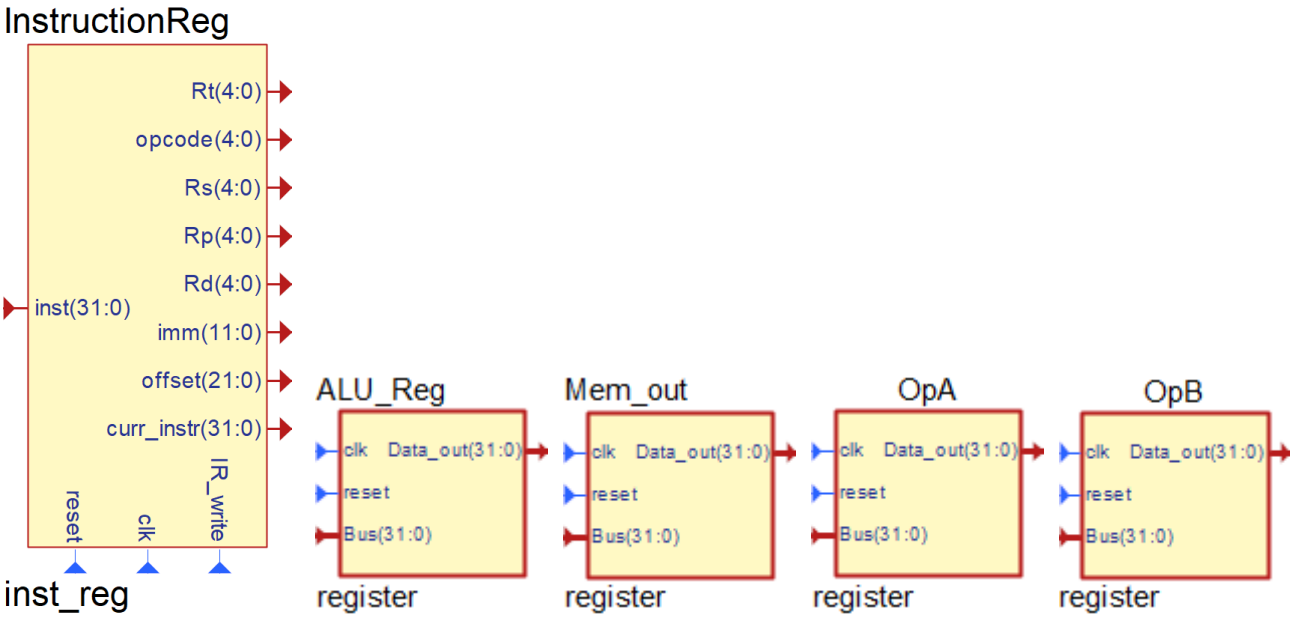


Figure 12 Rgister modules

Registers are versatile elements. The instruction register for one, takes the instruction from the instruction memory. Its job is to divide the instruction into the registers and other components like offset and immediate values. It does that according to the instruction formats we showed earlier. Another thing it does is save the value if the instruction it divides, which we used for simulation purposes. For the rest of the registers the main use was to save the values of the data stored in them in a synchronized way until we needed them.

3. Control Path

3.1. Control Signals Description

Reg_dest (1-bit): Determines whether the destination register is register 31 or Rd.

Reg_write (1-bit): enables writing on the register file.

Ext_op (1-bit): Determines whether the immediate value is sign extended or zero extended.

Mem_read (1-bit): enables reading from memory.

Mem_write (1-bit): enables writing on memory.

PC_write (1-bit): enables writing on PC in register file.

IR_write (1-bit): enables writing on the instruction register.

Reg_src (1-bit): determines whether to read register Rt or Rd.

ALU_srcB (1-bit): selects the second ALU source as BusB or the extended immediate.

ALU_ctrl (3-bit): controls the operation performed in the ALU.

Pc_src (2-bit): determines the source of the new PC value

WB_data (2-bit): determines source of the value to be written in the register file.

Table 2 Control signals effect when adderted and de-asserted

Signal	When de-asserted	When asserted
reg_dest	Destination register is Rd	Destination register is R31
reg_write	Disables writing on register file	Enables writing on register file
Ext_op	Immediate value is zero extended	Immediate value is sign extended
Mem_read	Reading from data memory is disabled	Reading from data memory is enabled
Mem_write	Writing on data memory is disabled	Writing on data memory is enabled
PC_write	Writing on PC is disabled	Writing on PC is enabled
IR_write	Writing on IR register is disabled	Writing on IR register is enabled
Reg_src	Rb in register file is Rt	Rb in register file is Rd
ALU_srcB	The second ALU input is OpB	The second ALU input is the sign extended immediate

Table 3 The effects of multi-bit control signals

Signal	Value	Effect
ALU_ctrl	000	Addition operation
	001	Subtraction operation
	010	OR operation
	011	NOR operation
	100	AND operation
PC_src	00	pc+1
	01	Jump address
	10	Reg[Rs]
WB_data	00	ALU result
	01	Data memory result
	10	PC value

1.1. Controller Truth Table

Table 4 Controller truth table

Inputs		**Outputs**												
Present State	Opcode	Next State	reg_dest	reg_src	reg_write	ext_op	WB_data	mem_read	mem_write	PC_write	PC_src	IR_write	ALU_srcB	ALU_ctrl
Fetch	X	Decode	x	0	0	x	xx	0	0	1	0	1	x	xxx
Decode	Load	address comp	x	0	0	1	xx	0	0	0	xx	0	x	xxx
Decode	Store	address comp	x	0	0	1	xx	0	0	0	xx	0	x	xxx
Decode	R-type	execute	x	0	0	x	xx	0	0	0	xx	0	x	xxx
Decode	I-type/A	execute	x	0	0	1	xx	0	0	0	xx	0	x	xxx
Decode	I-type/L	execute	x	0	0	0	xx	0	0	0	xx	0	x	xxx
Decode	call	wb	x	0	0	x	xx	0	0	0	xx	0	x	xxx
Decode	JR	jump	x	0	0	x	xx	0	0	0	xx	0	x	xxx
Decode	Jump	jump	x	0	0	x	xx	0	0	0	xx	0	x	xxx
address comp	Load	data_read	x	0	0	x	xx	0	0	0	xx	0	1	000
address comp	Store	data_write	x	0	0	x	xx	0	0	0	xx	0	1	000
execute	r-type	wb	x	0	0	x	xx	0	0	0	xx	0	0	**
execute	i-type	wb	x	0	0	x	xx	0	0	0	xx	0	1	**
wb	call	jump	1	0	1	x	10	0	0	0	xx	0	x	xxx
jump	JR	fetch	x	0	0	x	xx	0	0	1	10	0	x	xxx
jump	Jump	fetch	x	0	0	x	xx	0	0	1	01	0	x	xxx
data_read	Load	wb	x	0	0	x	xx	1	0	0	xx	0	x	xxx
data_write	Store	fetch	x	1	0	x	xx	0	1	0	xx	0	x	000

wb	r-type	fetch	0	0	1	x	00	0	0	0	xx	0	x	xxx
wb	i-type	fetch	0	0	1	x	00	0	0	0	xx	0	x	xxx
jump	call	fetch	x	0	0	x	xx	0	0	1	01	0	x	xxx
wb	Load	fetch	0	0	1	x	01	0	0	0	xx	0	x	xxx

1.2. Controller Boolean Expressions

Based on the values in table (), the following boolean expression were obtained:

$$Reg\_dest = (prstate == wb)\&\&(call)$$

$$Reg\_src = (prstate == data\_write)$$

$$Reg\_write = (prstate == wb)$$

$$ext\_op = (prstate == decode)\&\&(LW \,||\, SW \,||\, ADDI)$$

$$mem\_read = (prstate == data\_read)$$

$$mem\_write = (prstate == data\_write)$$

$$PC\_write = (prstate == fetch)||(prstate == jump)$$

$$IR\_write = (prstate == fetch)$$

$$ALU\_srcB = (prstate == address\_computation) \,||\, ((prstate == execute)\ \&\&$$
$$(\,(opcode == ADDI) \,||\, (opcode == ORI) \,||\, (opcode == NORI) \,||\, (opcode == ANDI)\,)\,)$$

$$WB\_data[1] = (prstate == wb)\&\&(call)$$

$$WB\_data[0] = (prstate == wb)\&\&(LW)$$

$$PC\_src[1] = (prstate == jump)\&\&(JR)$$

$$PC\_src[0] = (prstate == jump)\&\&(jump||call)$$

$$ALU\_ctrl[2] = (prstate == execution)\&\&(ANDI||AND)$$

$$ALU\_ctrl[1] = (prstate == execution)\&\&(NOR||OR||NORI||ORI)$$

$$ALU\_ctrl[0] = (prstate == execution)\&\&(SUB||NOR||NORI)$$

And the state encoding can be observed in the table below.

Table 5 State encodings

State	**Encoding**
fetch	3'b000
decode	3'b001
address_comp	3'b010
data_read	3'b011
data_write	3'b100
execute	3'b101
wb	3'b110
jump	3'b111

1.3. Controller Logic Diagram

Since a binary encoding was chosen for the states, a 3x8 decoder could be utilized for the controller's logic diagram. A decoder is combination circuit that takes n input lines and outputs a maximum of 2^n output lines. For every input, there is only one specific output line that is activated while all others remain inactive. So if the prstate is input to the decoder, a unique output port will be activated to indicate the prstate. Same logic can be applied to the opcode, but since the opcode is 5 bits, a larger decoder will be used. And according to this logic the following logic diagrams were drawn. The logic diagrams were drawn in two separate diagrams so that the logic can be observed clearly.

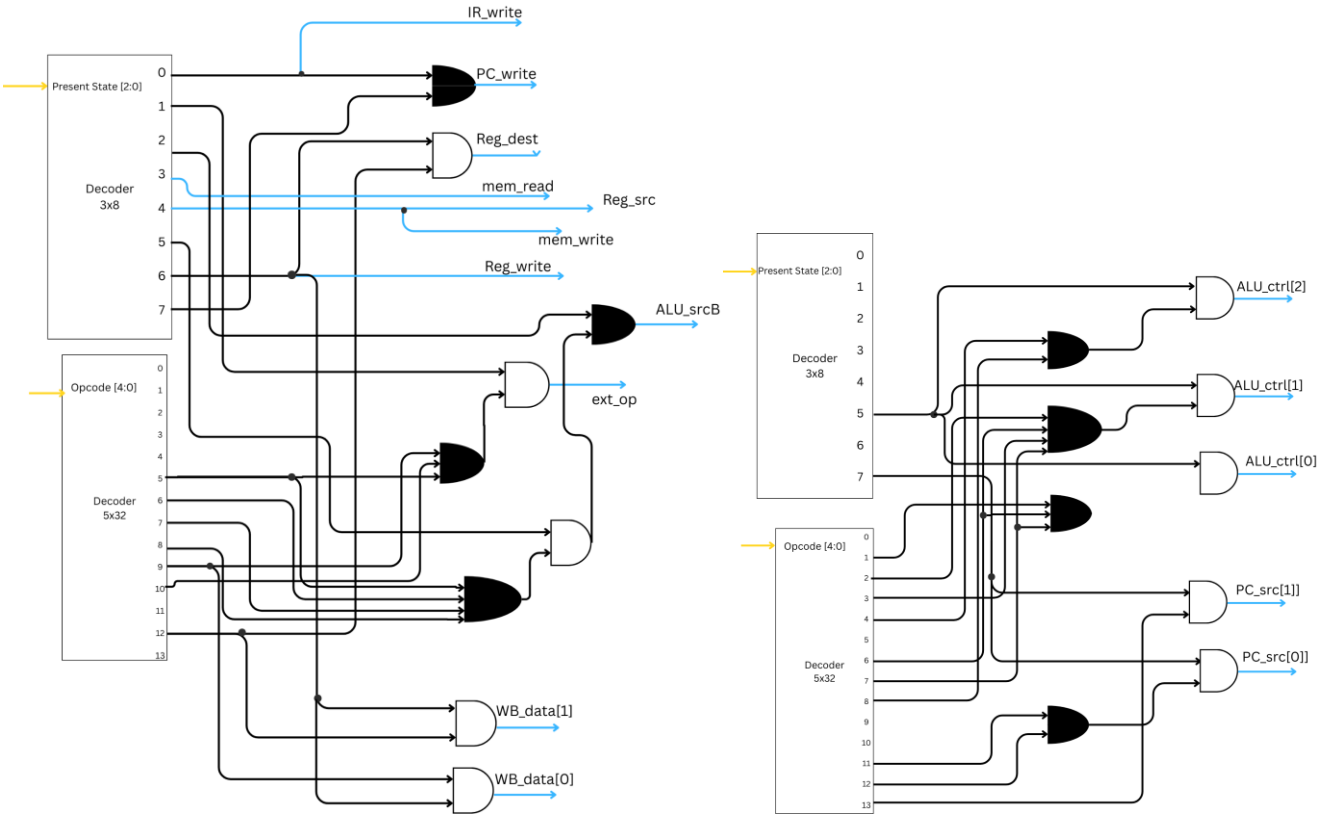


Figure 13 Controller logic diagrams

1.4. Controller State Diagram

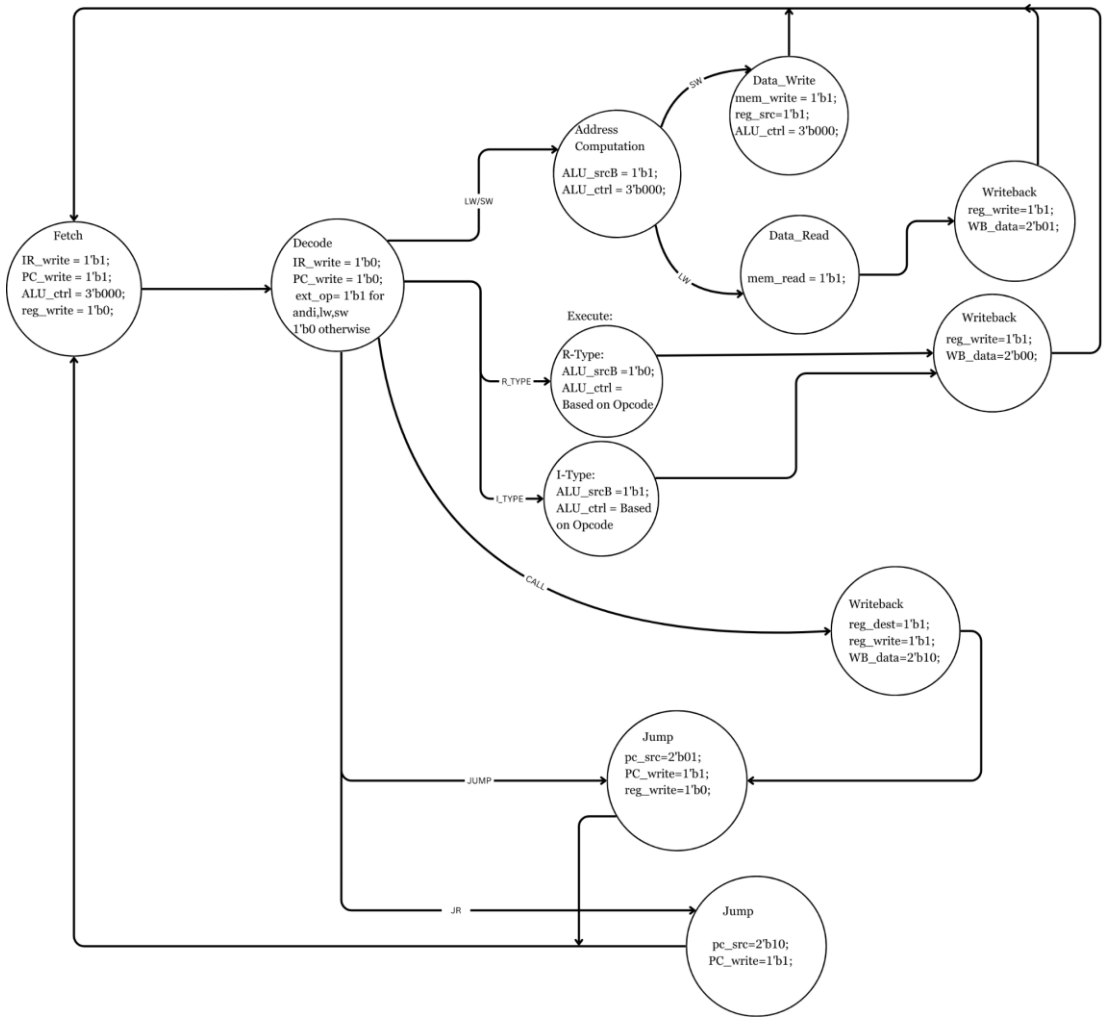


Figure 14 Controller state diagram

Every instruction undergoes the first two states: fetch and decode. So regardless of the opcode, every instruction starts at fetch and then transitions to decode.

In the fetch state, the processor prepares to read the next instruction, the instruction memory and increments the program counter. The IR_write is enabled, so the output of the instruction memory is loaded in the instruction register. Simultaneously, PC_src is zero by default thus selecting the incremented PC.

In the decode state, the extends the immediate value based on the opcode. If the opcode is LW, SW, or ADDI, the immediate is sign extended; otherwise it will be zero extended. Moreover, the PC_write and IR_write are disabled at this state to ensure correct sequencing of instructions. This phase also pre fetches the required registers for the execution phase.

Transitioning from the decode state to any other state is done based on opcode.

If the operation is:

- Store: following decode, a store operation would transition to the address computation state where the sign extended immediate would be taken as the second input into the ALU by activating ALU_srcB and the ALU_ctrl is set to the add operation in order to calculate the address in the data memory where the data is to be stored. Afterwhich, state transitions to the data_write state where the mem_write is enabled to allow writing on the data memory, reg_src is activated so Rd is read. And after the Data_write state, the program goes back to the fetch state and moves onto the next instruction.
  - State transitions of store operation: fetch → decode → address_comp → data_write → fetch
- Load: following decode, a load operation would transition to the address computation state where the sign extended immediate would be taken as the second input into the ALU by activating ALU_srcB and the ALU_ctrl is set to the add operation in order to calculate the address in the data memory where the data is to be stored. Afterwhich, the state transitions to the data_read state where the mem_read is enabled to allow reading of the data memory. And after the data_read state, the program transitions to the writeback state where the WB_data chooses to write the output of the memory on the register RW and reg_write is enabled to permit writing on the register. Afterwhich, it goes back to the fetch state and moves onto the next instruction.
  - State transitions of load operation: fetch → decode → address_comp → data_read → Writeback→fetch
- R-type: following decode, an R-type operation would transition to the execute state where the second ALU operand is chosen to be contents of the register Rt which have been stored in a register OpB and the ALU_ctrl are set based on the R-type operation. After the execution of the operation in the ALU, the program moves to the Writeback state. In the writeback state, as it is an R-type operation, the WB_data is set to take the ALU output data and reg_write is activated to enable writing on the register file.
  - State transitions of R-type operation: fetch → decode → execute→ writeback→ fetch
- I-type: following decode, an I-type operation would transition to the execute state where the second ALU operand is chosen to be the extended immediate value by activating ALU_srcB and

the ALU_ctrl are set based on the I-type operation. After the execution of the operation in the ALU, the program moves to the Writeback state. In the writeback state, as it is an non-load and non-store I-type operation, the WB_data is set to take the ALU output data and reg_write is activated to enable writing on the register file.

- ➢ State transitions of I-type operation: fetch → decode → execute→ writeback→ fetch

- ❖ Call: following decode, a call operation would transition to the writeback state where the reg_dest control signal would be activated to indicate that the register destination that will be written on is R31. Additionally, reg_write would be activated to enable writing on the register file and lastly the WB_data would be set to take the value of the PC to write on R31. Following the writeback state, the state will transition to the jump state. In the jump state, the pc_src will be set to take the jump address calculated in the jump calc module, the PC_write will be activated to enable writing on the PC and reg_write will be reset to zero. After the jump it will move back to the fetch state.

  - ➢ State transitions of R-type operation: fetch → decode → writeback→ jump→fetch

- ❖ Jump: following decode, a jump operation would transition the jump state. In the jump state, the pc_src will be set to take the jump address calculated in the jump calc module, the PC_write will be activated to enable writing on the PC. After the jump it will move back to the fetch state.

  - ➢ State transitions of R-type operation: fetch → decode → jump→fetch

- ❖ JR: following decode, a jump operation would transition the jump state. In the jump state, the pc_src will be set to take the contents of the register Rs which were stored in a register OpA, the PC_write will be activated to enable writing on the PC. After the jump it will move back to the fetch state.

  - ➢ State transitions of R-type operation: fetch → decode → jump→fetch

2. Full Datapath

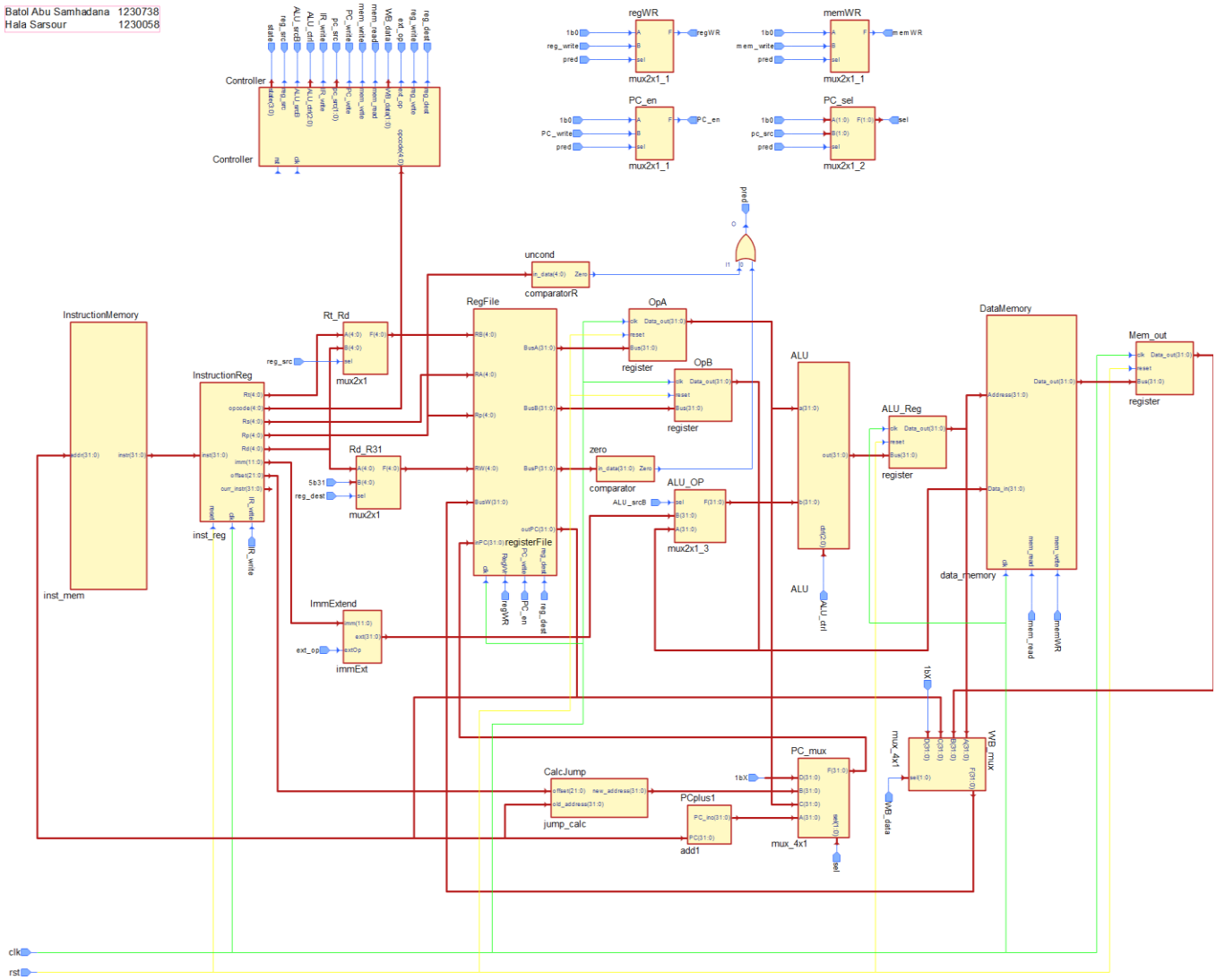


Figure 15 Full datapath

3. Testing

3.1. R-Type

```
83  // R-Type
84  memory[0] = 32'b00000000110000100010000110000000; // ADD  R1 = R2 + R3
85  memory[1] = 32'b00001000110000100010000110000000; // SUB  R1 = R2 - R3
86  memory[2] = 32'b00010000110000100010000110000000; // OR   R1 = R2 | R3
87  memory[3] = 32'b00011000110000100010000110000000; // NOR  R1 = ~(R2|R3)
88  memory[4] = 32'b00100000110000100010000110000000; // AND  R1 = R2 & R3
89  memory[5] = 32'b00000000000000000000000000000000; // NOP
```

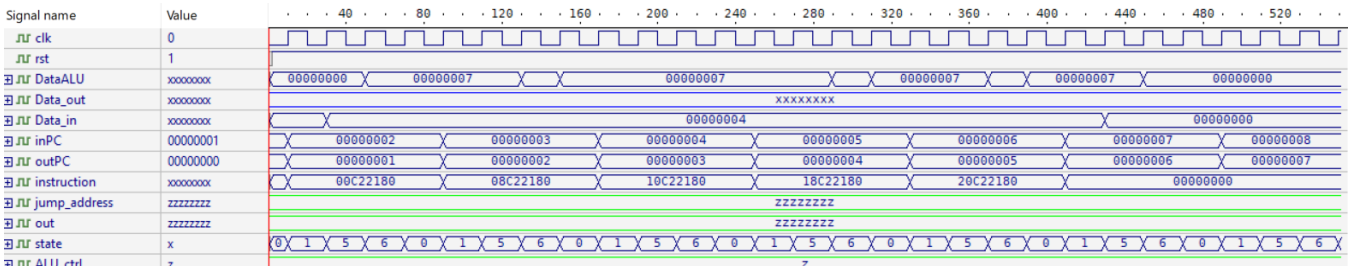


Figure 16 R-type testing Waveform

The waveform shows the correct state transitions for a sequence of multiple R-Type operations. Looking more closely at the waveform we can see the correct arithmetic and logical operations being performed by the ALU.

In the figure below, the ALU result has been highlighted during the execution state for:

ADD R3, R1, R2,R3

32'b00000000110000100010000110000000

Where the values stored in R2 and R3 are 3 and 4 respectively.

Thus the predicted output of this operation is 7 which can be seen clearly in the figure below.

Signal name	Value
clk	1
rst	1
DataALU	00000007
Data_out	xxxxxxxx
Data_in	00000004
inPC	00000002
outPC	00000001
instruction	00C22180
jump_address	zzzzzzzz
out	zzzzzzzz
state	6
ALU_ctrl	z

Figure 17 add test

SUB R3, R1, R2,R3

32'b00001000110000100010000110000000

Where the values stored in R2 and R3 are 3 and 4 respectively.

Thus the predicted output of this operation is -1 which can be seen clearly in the figure below.

Signal name	Value
clk	1
rst	1
DataALU	FFFFFFFF
Data_out	xxxxxxxx
Data_in	00000004
inPC	00000003
outPC	00000002
instruction	08C22180
jump_address	zzzzzzzz
out	zzzzzzzz
state	6
ALU_ctrl	z

Figure 18 subtract test

OR R3, R1, R2,R3

32'b00001000110000100010000110000000

Where the values stored in R2 and R3 are 3 and 4 respectively.

Thus the predicted output of this operation is 7 which can be seen clearly in the figure below.

Signal name	Value
clk	1
rst	1
DataALU	00000007
Data_out	xxxxxxxx
Data_in	00000004
inPC	00000004
outPC	00000003
instruction	10C22180
jump_address	zzzzzzzz
out	zzzzzzzz
state	6
ALU_ctrl	z

Figure 19 OR test

NOR R3, R1, R2,R3

32'b00011000110000100010000110000000

Where the values stored in R2 and R3 are 3 and 4 respectively.

Thus the predicted output of this operation is -8 which can be seen clearly in the figure below.

Signal name	Value
clk	1
rst	1
DataALU	FFFFFFF8
Data_out	xxxxxxxx
Data_in	00000004
inPC	00000005
outPC	00000004
instruction	18C22180
jump_address	zzzzzzzz
out	zzzzzzzz
state	6
ALU_ctrl	z

Figure 20 NOR test

AND R3, R1, R2,R3

32'b00100000110000100010000110000000

Where the values stored in R2 and R3 are 3 and 4 respectively.

Thus the predicted output of this operation is 0 which can be seen clearly in the figure below.

Signal name	Value
clk	1
rst	1
DataALU	00000000
Data_out	xxxxxxxx
Data_in	00000004
inPC	00000006
outPC	00000005
instruction	20C22180
jump_address	zzzzzzzz
out	zzzzzzzz
state	6
ALU_ctrl	z

Figure 21 AND test

Lastly, NoOp where no operation takes place

32'b00000000000000000000000000000000

Signal name	Value
clk	1
rst	1
DataALU	00000000
Data_out	xxxxxxxx
Data_in	00000000
inPC	00000007
outPC	00000006
instruction	00000000
jump_address	zzzzzzzz
out	zzzzzzzz
state	6
ALU_ctrl	z

Figure 22 NO OP test

3.2. I-Type

```
83      // I-Type
84      memory[0] = 32'b00101000110000100010000000000101; // ADDI R1 = R2 + 5
85      memory[1] = 32'b00110000110000100010000000001111; // ORI  R1 = R2 | 0xF
86      memory[2] = 32'b00111000110000100010000000001111; // NORI R1 = ~(R2 | 0xF)
87      memory[3] = 32'b01000000110000100010000000001111; // ANDI R1 = R2 & 0xF
88      memory[4] = 32'b00000000000000000000000000000000; // NOP
```

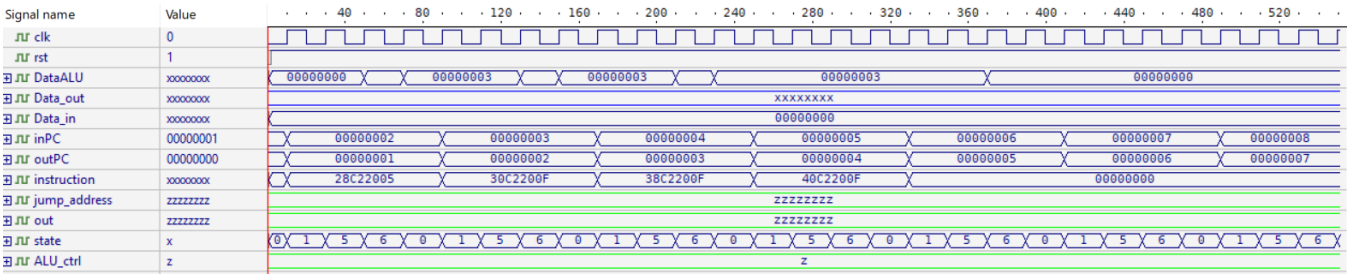


Figure 23 I-type testing Waveform

The waveform shows the correct state transitions for a sequence of multiple I-Type operations. Looking more closely at the waveform we can see the correct arithmetic and logical operations being performed by the ALU.

In the figure below, the ALU result has been highlighted during the execution state for:

ADDI R3, R1, R2, 5

32'b00101_00011_00001_00010_000000000101

Where the value stored in R2 is 3.

Thus the predicted output of this operation is 8 which can be seen clearly in the figure below.

Signal name	Value
clk	1
rst	1
DataALU	00000008
Data_out	xxxxxxxx
Data_in	00000000
inPC	00000002
outPC	00000001
instruction	28C22005
jump_address	zzzzzzzz
out	zzzzzzzz
state	6
ALU_ctrl	z

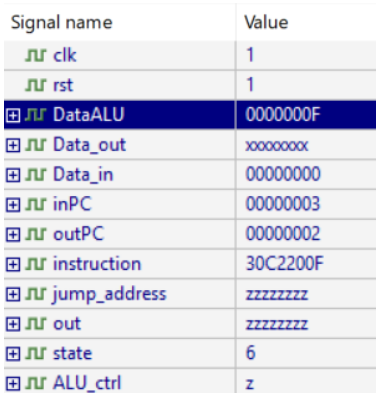
Figure 24 ADDI test

ORI R3, R1, R2, 0xF

32'b00110_00011_00001_00010_000000001111

Where the value stored in R2 is 3.

Thus the predicted output of this operation is 0xF which can be seen clearly in the figure below.



Signal name	Value
clk	1
rst	1
DataALU	0000000F
Data_out	xxxxxxxx
Data_in	00000000
inPC	00000003
outPC	00000002
instruction	30C2200F
jump_address	zzzzzzzz
out	zzzzzzzz
state	6
ALU_ctrl	z

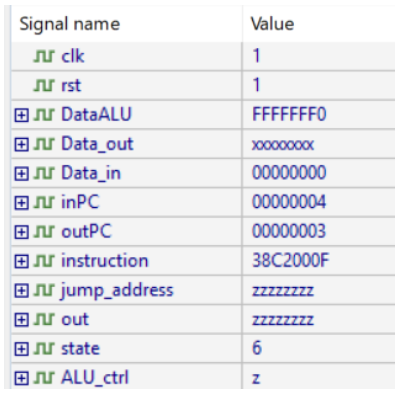
Figure 25 ORI test

NORI R3, R1, R2, 0xF

32'b00111_00011_00001_00010_000000001111

Where the value stored in R2 is 3.

Thus the predicted output of this operation is -16 which can be seen clearly in the figure below.



Signal name	Value
clk	1
rst	1
DataALU	FFFFFFF0
Data_out	xxxxxxxx
Data_in	00000000
inPC	00000004
outPC	00000003
instruction	38C2000F
jump_address	zzzzzzzz
out	zzzzzzzz
state	6
ALU_ctrl	z

Figure 26 NORI test

ANDI R3, R1, R2, 0xF

32'b01000_00011_00001_00010_000000001111

Where the value stored in R2 is 3.

Thus the predicted output of this operation is 3 which can be seen clearly in the figure below.

Signal name	Value
clk	1
rst	1
DataALU	00000003
Data_out	xxxxxxxx
Data_in	00000000
inPC	00000005
outPC	00000004
instruction	40C2200F
jump_address	zzzzzzzz
out	zzzzzzzz
state	6
ALU_ctrl	z

Figure 27 ANDI test

3.3. Memory Instructions:

```
83      // Memory Instructions
84      memory[0] = 32'b01001111000010000010000000000000; // LW R4, 107(R0)
85      memory[1] = 32'b01010000000010100100000000000000; // SW R5, 0(R4)
86      memory[2] = 32'b00000000000000000000000000000000; // NOP
```

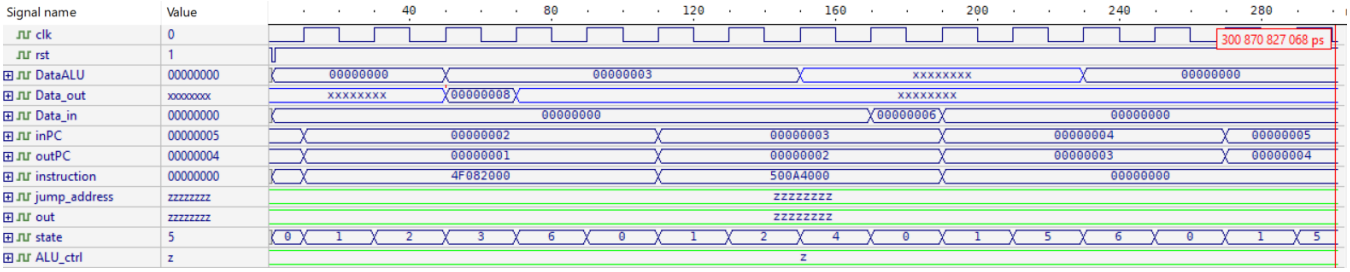


Figure 28 Memory instructions testing Waveform

The waveform shows the correct state transitions for a sequence of multiple memory instruction operations. Looking more closely at the waveform we can see the correct expected data coming into and out of the data memory

In the figure below, the Data_in and Data_out results have been highlighted during their respective states for:

LW R28, R4, 0(R2)

32'b01001_11100_00100_00010_000000000000

Where the value stored in R2 is 3 and value stored in mem[3] is 8.

Thus, the predicted data_out which is 8 can be seen clearly in the figure below.

Signal name	Value
clk	0
rst	1
DataALU	00000003
Data_out	00000008
Data_in	00000000
inPC	00000002
outPC	00000001
instruction	4F082000
jump_address	zzzzzzzz
out	zzzzzzzz
state	3
ALU_ctrl	z

Figure 29 LW test

SW R28, R5, 0(R2)

32'b01010_00000_00101_00100_000000000000

Where the value stored in R2 is 6 and from the previous instruction R2 has become 8.

Thus the predicted data_in of 6 can be seen clearly in the figure below.

Signal name	Value
clk	1
rst	1
DataALU	xxxxxxxx
Data_out	xxxxxxxx
Data_in	00000006
inPC	00000003
outPC	00000002
instruction	500A4000
jump_address	zzzzzzzz
out	zzzzzzzz
state	0
ALU_ctrl	z

Figure 30 SW test

3.4. Jump and Control flow

```
83          // Jump and Control flow
84          memory[0] = 32'b01011000000000000000000000000010; // J +2
85          memory[1] = 32'b00000000000000000000000000000000; // skipped
86          memory[2] = 32'b01100000000000000000000000000011; // CALL
87          memory[3] = 32'b01101000000000011111000000000000; // JR R1
```

The jump instructions were tested to verify correct program counter updates and control signal behavior. The following J-Type and R-Type jump instructions were used during simulation.

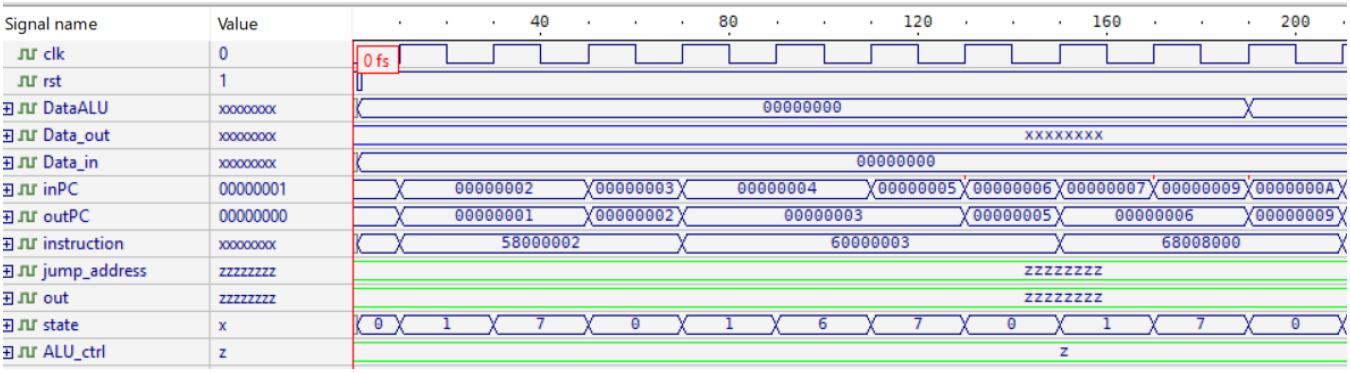


Figure 31 J-type and Control flow testing Waveform

Signal name	Value
clk	1
rst	1
DataALU	00000000
Data_out	xxxxxxxx
Data_in	00000000
inPC	00000002
outPC	00000001
instruction	58000002
jump_address	zzzzzzzz
out	zzzzzzzz
state	7
ALU_ctrl	z

Figure 32 J test

For the jump instruction **J**:

J PC+2, R0

32'b01011_00000_00000000000000000000010

When the predicate register is R0 (unconditional), the processor transitions from the decode to the jump state directly. In this state PC_src selects the jump address generated by the jump_calc module and PC_wite is asserted. The figure confirms that the PC is updated correctly from 1 to 3.

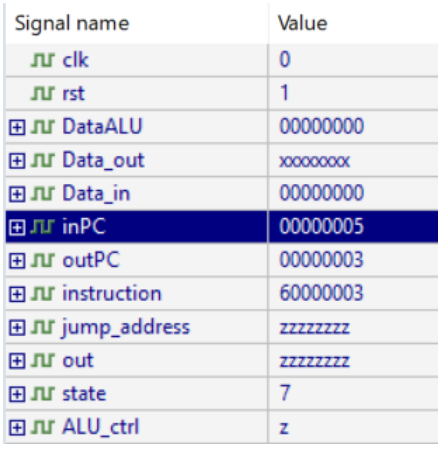


Figure 33 CALL test

For the **CALL** instruction:

CALL PC+3, R0

32'b01100_00000_00000000000000000000011

The processor first transitions to the writeback state, where the return address (PC+1) is stored in register R31. The processor then enters the jump state, updating the program counter (3) to the target address (5). The figure confirms correct saving of the return address and correct control flow.

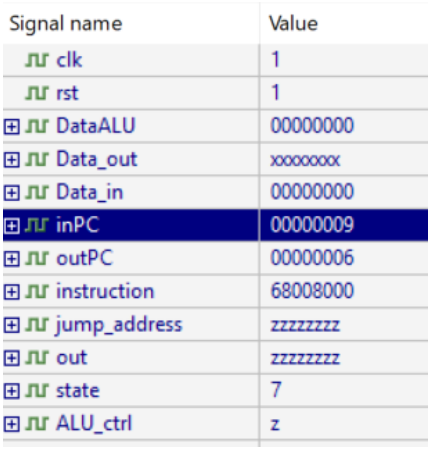


Figure 34JR test

For the **JR** instruction:

JR R8, R0

32'b01101_00000_01000_00000_000000000000

The jump target is taken from the contents or the source register Rs. In the jump state, PC_src selects Reg[R8] and PC_write is asserted. The waveform shows that the execution resumes from the address stored in the register (9 which is the value inside R[8]).

3.5. Predicated Execution

```
// Predicated Execution
    memory[0] = 32'b00000_00101_00001_00010_00011_0000000; // does not execute
    memory[1] = 32'b00000_00000_00001_00001_00010_0000000; // shows that it hasnt executed
```

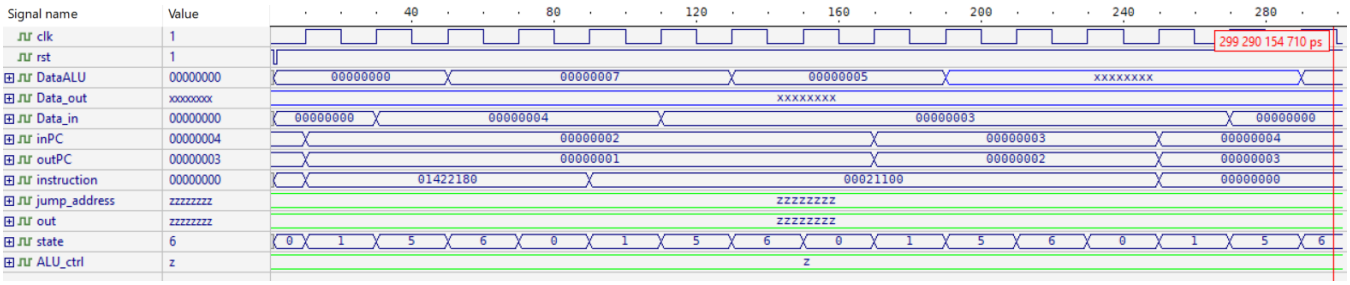


Figure 35 Predicated execution testing Waveform

In this sequence of execution, the following instructions are performed:

ADD R5, R1,R2, R3

32'b00000_00101_00001_00010_00011_0000000

ADD R0, R1,R1,R2

32'b00000_00000_00001_00001_00010_0000000

The register file starts with the values below:

R1=2

R2=3

R3=4

R5=0

So, while the first instruction performs an additional operation ADD R5, R1,R2, R3, the ALU output of which is expected to be 7, the result is not written on the R1. This is because the predicate conditions fail since R5 is zero. In the second operation performed, ADD R0, R1,R1,R2. The result of the ALU execution observed in DataALU is 5 which means that no change has taken place in R1 following the first instruction. This verifies that the predicate logic in our design works correctly.

Conclusion

In this project we presented the design of a simplified predicated RISC processor implemented in Verilog. The processor follows a RISC-based architecture with separate memories for data and instructions. It also supports predicated execution using a predicate register Rp, which allows conditional instruction execution.

We also designed a complete datapath that supports R-Type, I-Type and J-Type instructions, with all components implemented in an organized modular way. And we built the control path using a FSM, along with truth tables, Boolean expressions and logic diagrams, to generate the control signals necessary for each instruction and stage.

The processor was verified through a comprehensive testbench that includes arithmetic, logical, memory, jump and predicated instructions. Simulation results and waveforms confirmed correct state transitions, data flow and control signal behavior. Overall this project taught us a lot about processor datapath design, control logic, and RTL design verification for a multicycle RISC processor.

References:

[1]: MultiCycle von neumann risc implementation code on github
https://github.com/Hola39e/MIPS_Multi_Implementation/blob/master/MIPS_Multi_Cycle/Control_Unit.v#L33